Reg No: 2303917720522079

Exp. No : **Implementation of a Simple Resource Allocation Algorithm on Cloudsim**

Date : /9/2025

Aim:

To implement a simple resource allocation algorithm in a cloud scenario simulation using cloudsim

Pre-requisites:

Java Development Kit (JDK), Eclipse/IntelliJ, CloudSim libraries

Description:

- Implements a basic resource allocation policy in CloudSim.
- Uses First-Come-First-Serve (FCFS) strategy for VM allocation.
- Shows how to create datacenters, brokers, VMs, and cloudlets.
- Demonstrates compiling and running simulation code in Eclipse.
- Provides Java code for basic resource allocation logic.
- Explains how to link commons-math and CloudSim libraries.
- Illustrates how to submit VM and cloudlet lists to the broker.
- Simulates VM execution and prints cloudlet results.
- Offers a hands-on approach to understand cloud resource allocation.
- Confirms successful execution of the resource allocation algorithm.

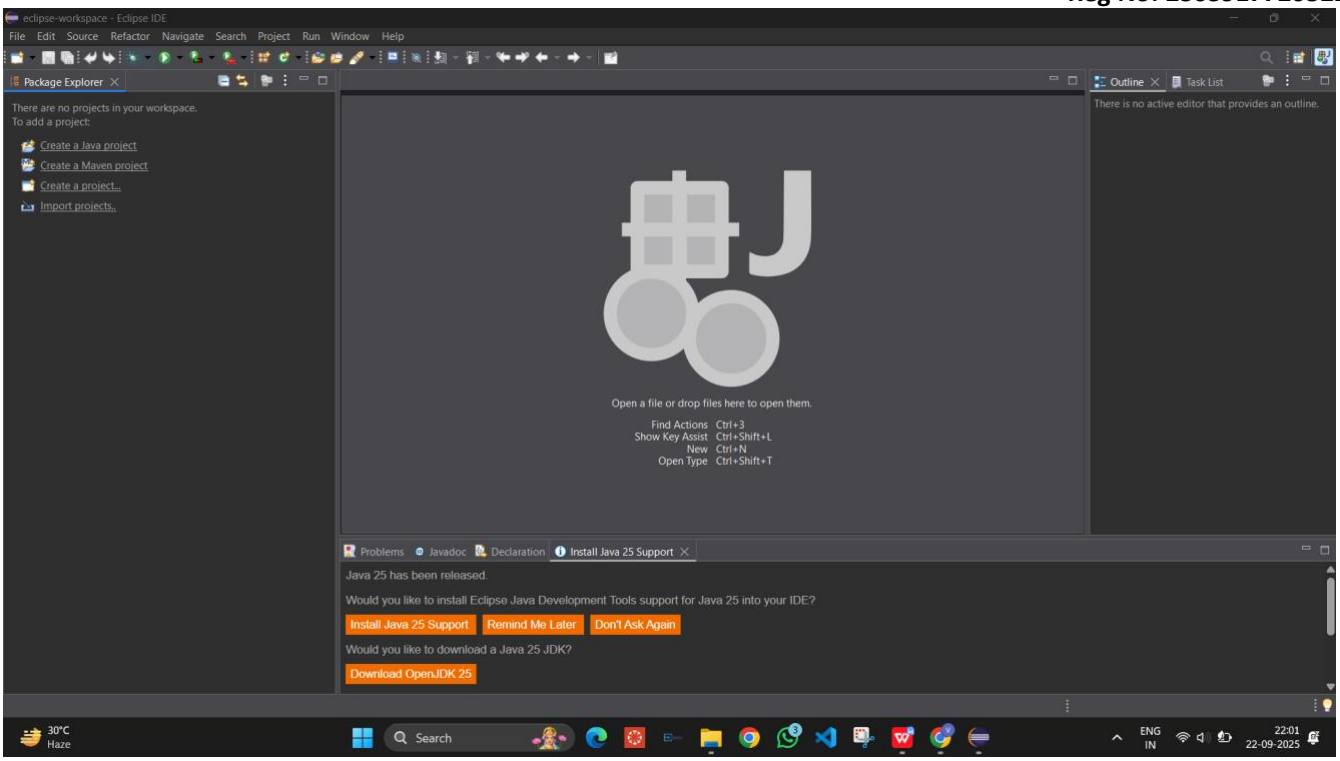
Procedure:

Install Eclipse and Keep the IDE ready

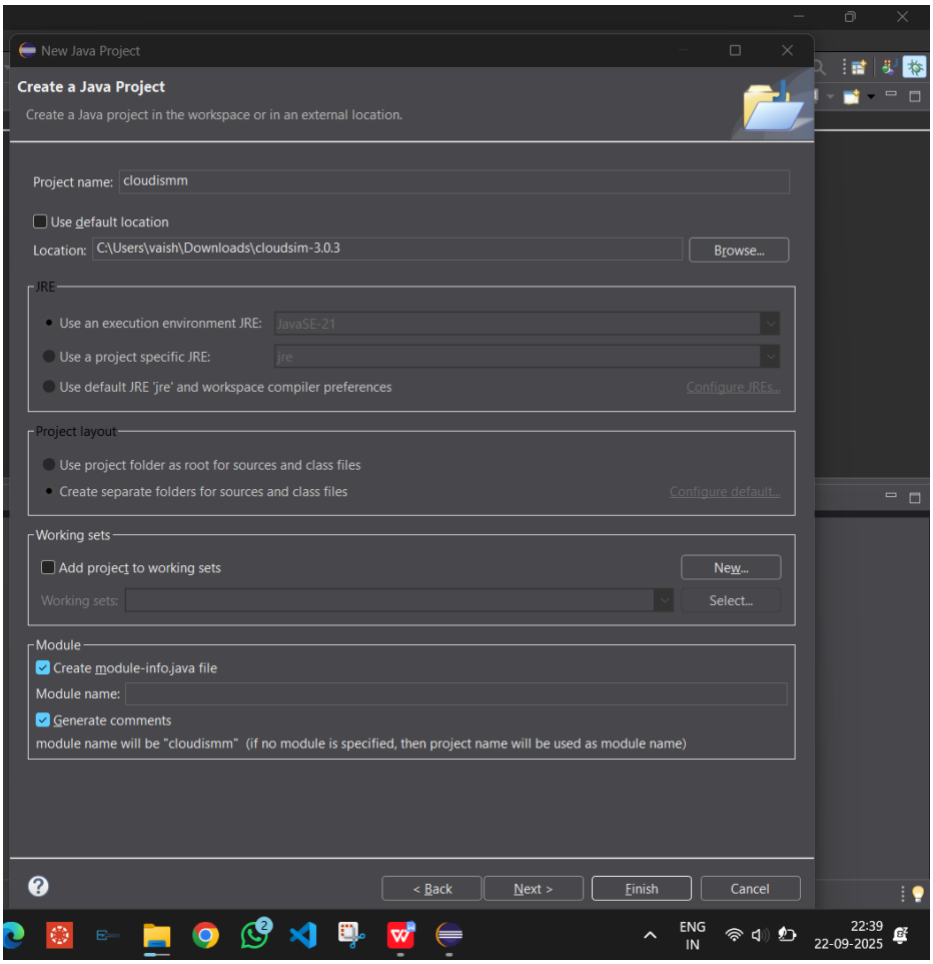
Download Cloudsim 3.0.3 from https://github.com/Cloudslab/cloudsim/releases/tag/cloudsim-3.0.3 and unzip.

Download commons-math3-3.6.1-sources.jar from https://repo.maven.apache.org/maven2/org/apache/commons/commons-math3/3.6.1/ and unzip

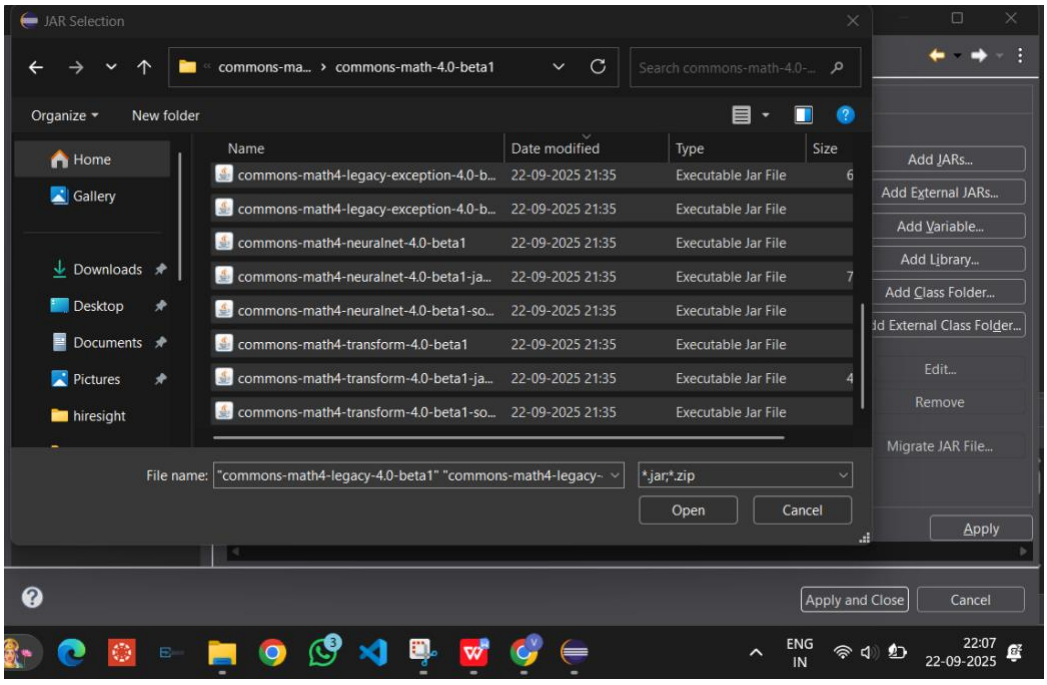
Open Eclipse. Click File > New > Java Project



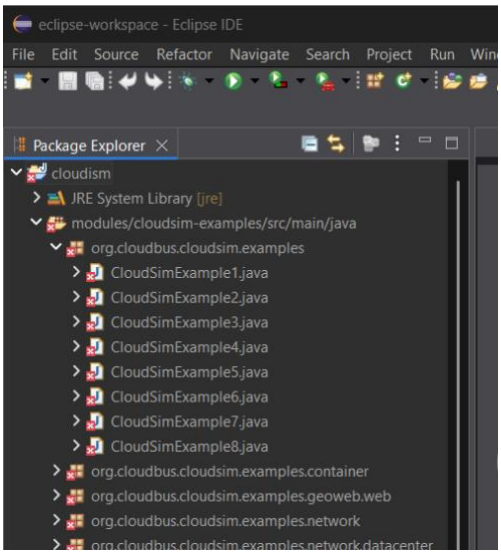
Give name for the project as ‘cloudsim’ and set the location to the path containing Cloudsim3.0.3



Select Java Build Path > Libraries > External Jars

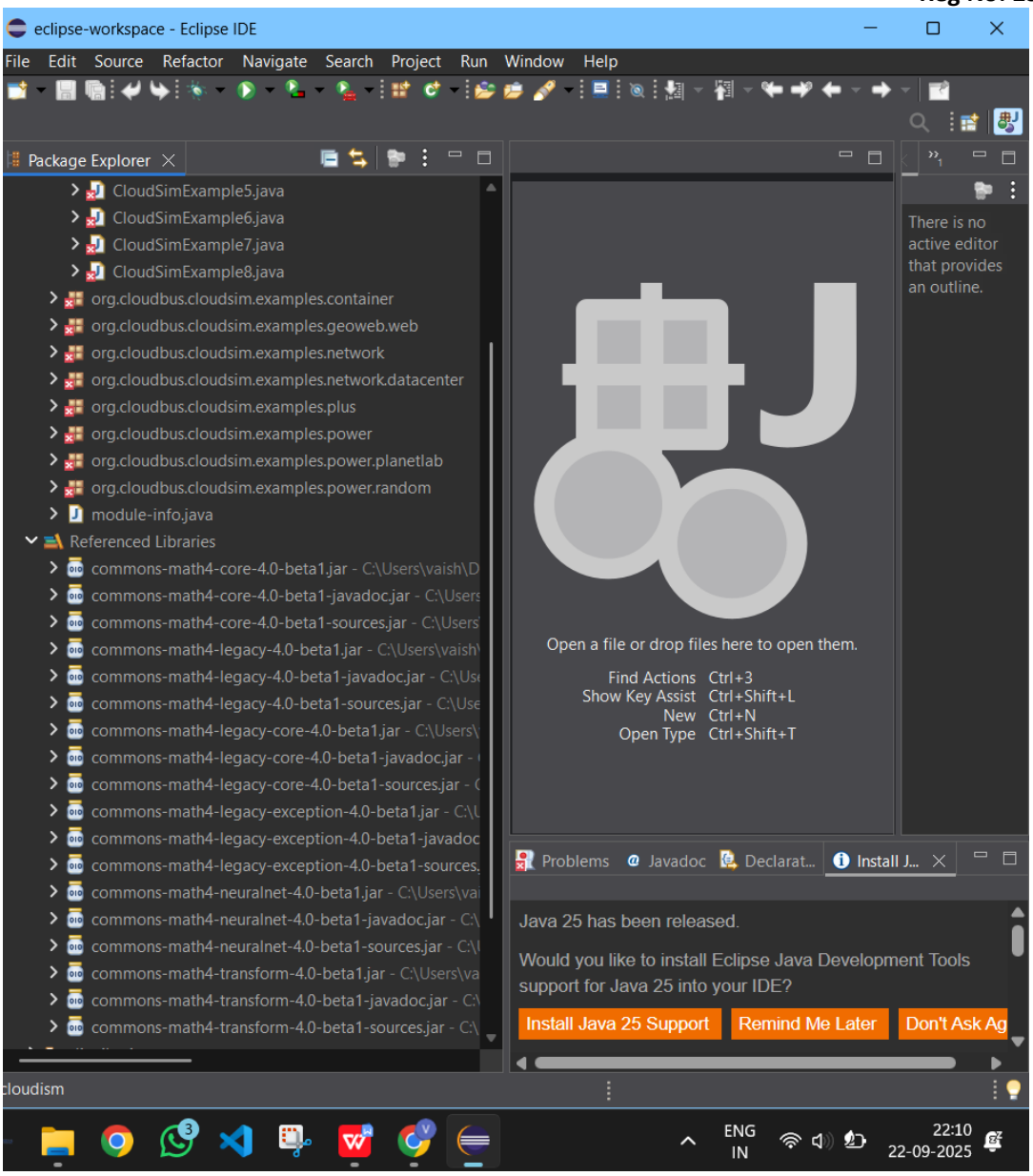


If you expand the cloudsim project, you will be able to view example java files



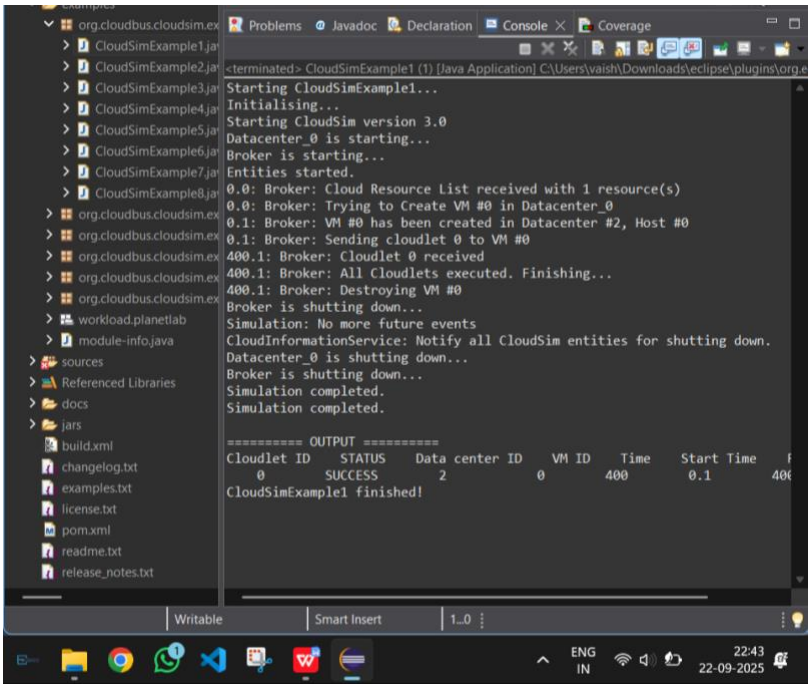
The Jar files added may be found under referenced libraries.

Reg No: 2303917720522079

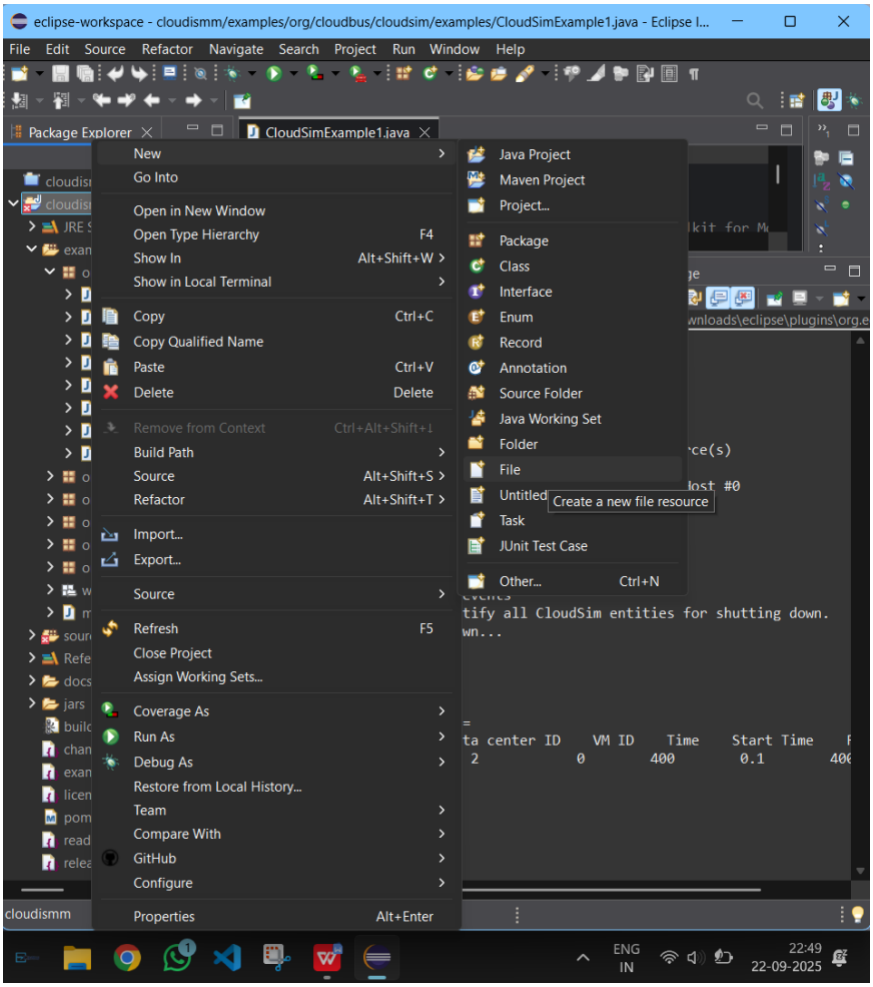


Reg No: 2303917720522079

Execute a sample program to ensure the installation is working fine

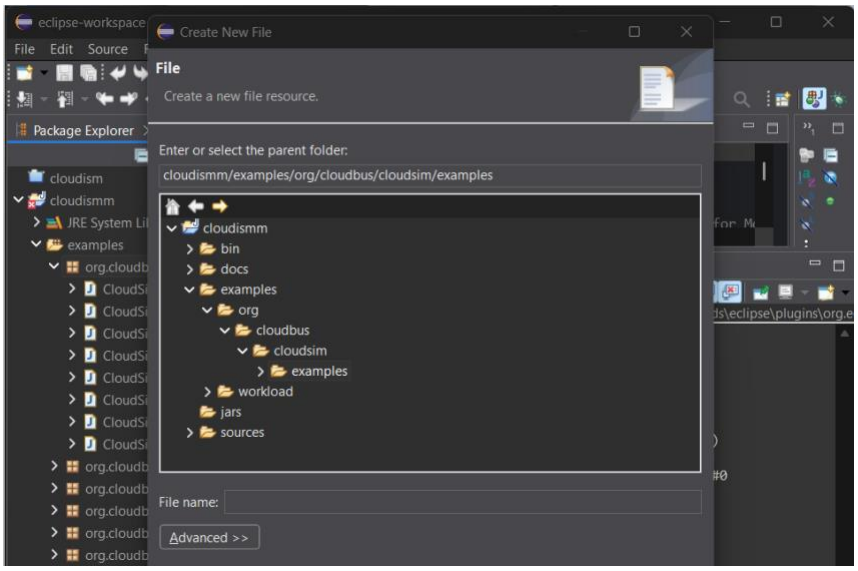


Right Click on the package ‘org.cloudbus.cloudsim.examples, New > File



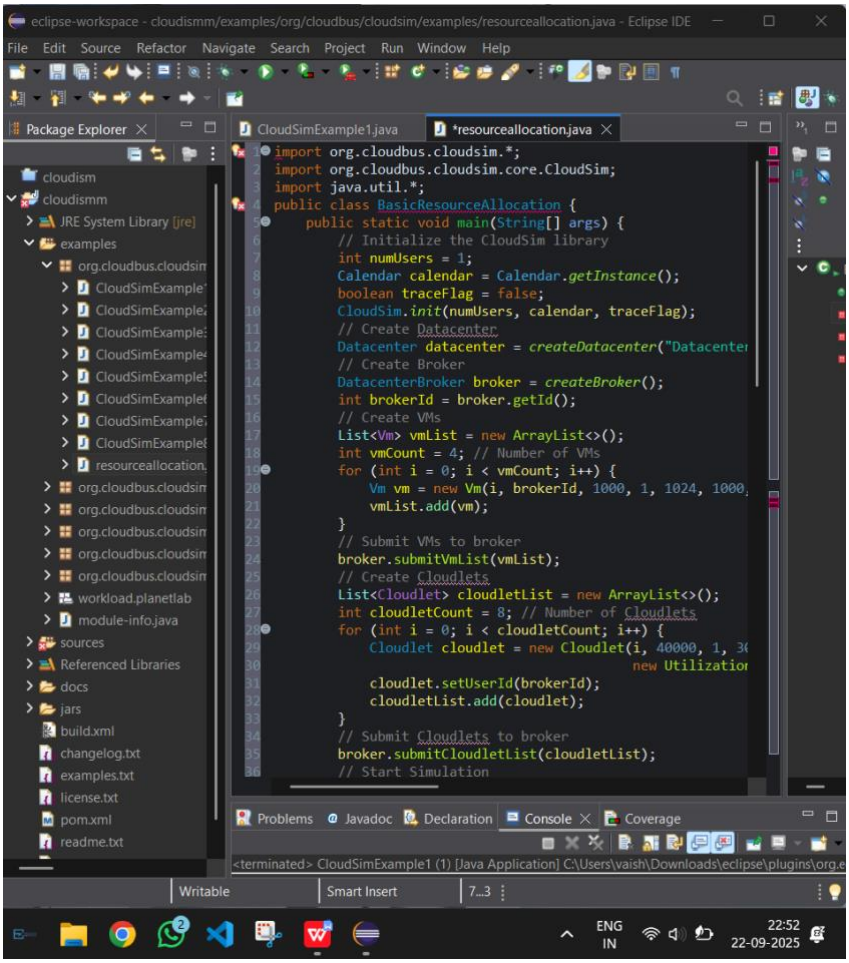
Reg No: 2303917720522079

Create a new file named “resourceallocation.java”

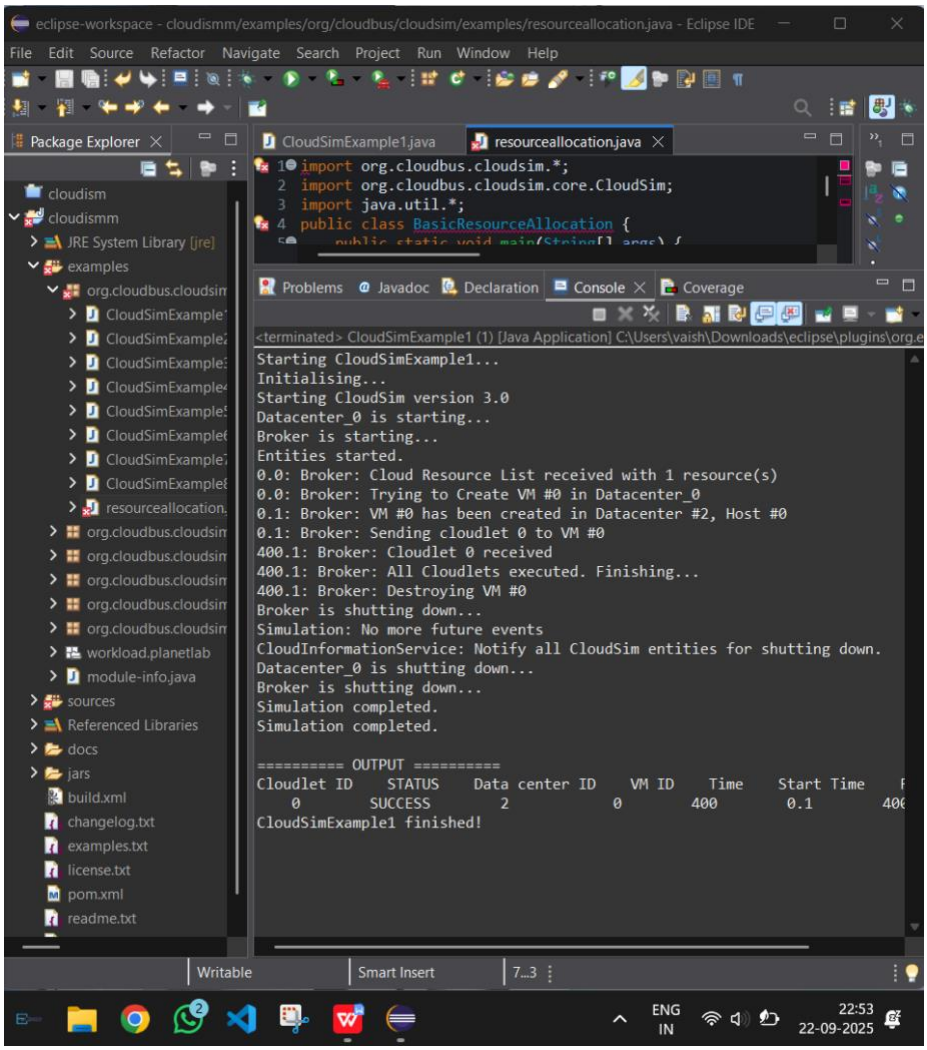


Add the code :

The basic idea is to create a simple allocation policy where Virtual Machines (VMs) are allocated to the hosts based on available resources. This can be a First-Come-First-Serve (FCFS) policy, where the first VM request is served by the first available host that meets its requirements.



Compile the code and Run



Code :

```
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import java.util.*;
public class BasicResourceAllocation {
   public static void main(String[] args) {
      // Initialize the CloudSim library
      int numUsers = 1;
      Calendar calendar = Calendar.getInstance();
      boolean traceFlag = false;
      CloudSim.init(numUsers, calendar, traceFlag);
      // Create Datacenter
      Datacenter datacenter = createDatacenter("Datacenter_0");
      // Create Broker
      DatacenterBroker broker = createBroker();
      int brokerId = broker.getId();
```

```
    // Create VMs
    List<Vm> vmList = new ArrayList<>();
    int vmCount = 4; // Number of VMs
    for (int i = 0; i < vmCount; i++) {
      Vm vm = new Vm(i, brokerId, 1000, 1, 1024, 1000, 10000, "Xen", new
CloudletSchedulerTimeShared());
      vmList.add(vm);
    }
    // Submit VMs to broker
    broker.submitVmList(vmList);
    // Create Cloudlets
    List<Cloudlet> cloudletList = new ArrayList<>();
    int cloudletCount = 8; // Number of Cloudlets
    for (int i = 0; i < cloudletCount; i++) {
      Cloudlet cloudlet = new Cloudlet(i, 40000, 1, 300, 300, new UtilizationModelFull(),
                    new UtilizationModelFull(), new UtilizationModelFull());
      cloudlet.setUserId(brokerId);
      cloudletList.add(cloudlet);
    }
    // Submit Cloudlets to broker
    broker.submitCloudletList(cloudletList);
    // Start Simulation
    CloudSim.startSimulation();
    // Stop Simulation
    CloudSim.stopSimulation();
    // Print results
    List<Cloudlet> resultList = broker.getCloudletReceivedList();
    printCloudletResults(resultList);
  }
  private static Datacenter createDatacenter(String name) {
    List<Host> hostList = new ArrayList<>();
    int hostCount = 4; // Number of Hosts
    for (int i = 0; i < hostCount; i++) {
      int hostId = i;
      int ram = 2048; // Host RAM
      long storage = 1000000; // Host storage
      int bw = 10000; // Host bandwidth

      List<Pe> peList = new ArrayList<>();
      peList.add(new Pe(0, new PeProvisionerSimple(1000))); // Host CPU cores
      Host host = new Host(hostId, new RamProvisionerSimple(ram), new BwProvisionerSimple(bw),
storage, peList, new VmSchedulerTimeShared(peList));
      hostList.add(host);
    }
    String arch = "x86"; // architecture
    String os = "Linux"; // operating system
    String vmm = "Xen"; // virtual machine monitor
```

```
    double time_zone = 10.0; // time zone this resource located
    double cost = 3.0; // the cost of using processing in this resource
    double costPerMem = 0.05; // the cost of using memory in this resource
    double costPerStorage = 0.001; // the cost of using storage in this resource
    double costPerBw = 0.0; // the cost of using bw in this resource
    DatacenterCharacteristics characteristics = new DatacenterCharacteristics(arch, os, vmm, hostList,
time_zone,
        cost, costPerMem, costPerStorage, costPerBw);
    Datacenter datacenter = null;
    try {
      datacenter = new Datacenter(name, characteristics, new VmAllocationPolicySimple(hostList), new
LinkedList<>(), 0);
    } catch (Exception e) {
      e.printStackTrace();
    }
    return datacenter;
  }
  private static DatacenterBroker createBroker() {
    DatacenterBroker broker = null;
    try {
      broker = new DatacenterBroker("Broker");
    } catch (Exception e) {
      e.printStackTrace();
    }
    return broker;
  }
  private static void printCloudletResults(List<Cloudlet> list) {
    String indent = "    ";
    System.out.println("Cloudlet ID" + indent + "STATUS" + indent + "Data center ID" + indent + "VM ID"
+ indent + "Time" + indent + "Start Time" + indent + "Finish Time");
    for (Cloudlet cloudlet : list) {
      System.out.print(indent + cloudlet.getCloudletId() + indent + indent);
      if (cloudlet.getStatus() == Cloudlet.SUCCESS) {
        System.out.println("SUCCESS" + indent + indent + cloudlet.getResourceId() + indent + indent +
cloudlet.getVmId() + indent + indent
            + cloudlet.getActualCPUTime() + indent + cloudlet.getExecStartTime() + indent + indent +
cloudlet.getFinishTime());}
    }
  }
}
```

Result:

Thus, the Implementation of a Simple Resource Allocation Algorithm on Cloudsim has been completed successfully.